YZM205 · HAFTA 6

NESNE YÖNELİMLİ PROGRAMLAMA LAB.

EĞİTMEN

Prof. Dr. Zafer CÖMERT

zcomert@samsun.edu.tr

YARDIMCI EĞİTMEN

Arş. Gör. Furkancan DEMİRCAN

furkancan.demircan@samsun.edu.tr

İçindekiler

Amaç ve Kapsam

01 Abstract Temel Sınıfın ve Akademik Birimin Oluşturulması

02 Erişim Belirleyicileri ve Ortak Hesaplama Metodu

03 İdari Birimin Oluşturulması

04 University Sınıfıyla Üniversite Raporunun Oluşturulması

Kazanımlar

Amaç ve Kapsam

Problem: Bu uygulamanın amacı, bir üniversitenin akademik ve idari birimlerini ortak bir abstract sınıf üzerinden modellemek ve sınıf üyelerine erişimi uygun erişim belirleyicileriyle düzenlemektir. Birimlerin ortak özellikleri **UniversityUnit** adlı abstract sınıfta tutulacaktır. Akademik birimler **AcademicUnit**, idari birimler ise **AdministrativeUnit** sınıfıyla temsil

edilecektir. Üniversiteye ait birimler **University** sınıfında bir araya getirilecektir.

→ GEREKSİNİMLER / ADIMLAR

1. Her birimin numarası, adı, personel sayısı ve personel başına yıllık maliyeti bulunmalıdır.
2. Birim numarası nesne oluşturulduktan sonra değiştirilememelidir.
3. Birim adı boş veya yalnızca boşluklardan oluşan bir metin olamaz.
4. Personel sayısı, öğrenci sayısı ve hizmet sayısı negatif olamaz.
5. Maliyet ve bütçe değerleri negatif olamaz ve decimal türünde tutulmalıdır.
6. Geçersiz güncellemelerde mevcut değer korunmalıdır.
7. Birim numaraları Main içerisinde pozitif ve birbirinden farklı değerlerle verilmelidir.
8. Parasal değerler iki ondalık basamakla gösterilmelidir.

Harici çizim kapalı. Diyagram için düzenleyicide PlantUML iznini açın. UML kodu:

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false

abstract class UniversityUnit {
    - unitId : int
    - unitName : string
    - staffCount : int
    - annualCostPerStaff : decimal

    + UnitId : int <<get only>>
    + UnitName : string <<property>>
    + StaffCount : int <<property>>
    + AnnualCostPerStaff : decimal <<property>>

    # UniversityUnit(inputUnitId : int, inputUnitName : string,
inputStaffCount : int, inputAnnualCostPerStaff : decimal)

    + DisplayInfo() : void
    # CalculateAnnualPersonnelCost() : decimal
    {abstract} + CalculateAnnualBudget() : decimal
}

class AcademicUnit {
    - studentCount : int
```

```
    - researchBudget : decimal

    + StudentCount : int <<property>>
    + ResearchBudget : decimal <<property>>

    + AcademicUnit(inputUnitId : int, inputUnitName : string,
inputStaffCount : int, inputAnnualCostPerStaff : decimal,
inputStudentCount : int, inputResearchBudget : decimal)

    + CalculateAnnualBudget() : decimal <<override>>
}

class AdministrativeUnit {
    - serviceCount : int

    + ServiceCount : int <<property>>

    + AdministrativeUnit(inputUnitId : int, inputUnitName :
string, inputStaffCount : int, inputAnnualCostPerStaff :
decimal, inputServiceCount : int)

    + CalculateAnnualBudget() : decimal <<override>>
}

class University {
    - universityName : string
    - units : UniversityUnit[]

    + University(inputUniversityName : string, inputUnits :
UniversityUnit[])

    + CalculateTotalBudget() : decimal
    + CalculateTotalStaff() : int
    + DisplayReport() : void
}

UniversityUnit <|-up- AcademicUnit
UniversityUnit <|-down- AdministrativeUnit
UniversityUnit : units
@enduml
```

01 Abstract Temel Sınıfın ve Akademik Birimin Oluşturulması

Üniversite birimlerinin ortak özelliklerini temsil etmek amacıyla **UniversityUnit** adlı abstract bir sınıf oluşturunuz.

Harici çizim kapalı. Diyagram için düzenleyicide PlantUML iznini açın. UML kodu:

```
abstract class UniversityUnit {
    - unitId : int
    - unitName : string
    - staffCount : int
    - annualCostPerStaff : decimal

    + UnitId : int <<get only>>
    + UnitName : string <<property>>
    + StaffCount : int <<property>>
    + AnnualCostPerStaff : decimal <<property>>

    # UniversityUnit(inputUnitId : int, inputUnitName : string,
inputStaffCount : int, inputAnnualCostPerStaff : decimal)

    + DisplayInfo() : void
    # CalculateAnnualPersonnelCost() : decimal
    {abstract} + CalculateAnnualBudget() : decimal
}
```

BEKLENEN ÇIKTI

terminal

```
AKADEMİK BİRİM BİLGİLERİ

Birim numarası: 101
Birim adı: Mühendislik Fakültesi
Personel sayısı: 10
Personel başına yıllık maliyet: 600.000,00 TL
Öğrenci sayısı: 200
Araştırma bütçesi: 500.000,00 TL

Yıllık bütçe: 6.700.000,00 TL
```

02 Erişim Belirleyicileri ve Ortak Hesaplama Metodu

UniversityUnit sınıfına **CalculateAnnualPersonnelCost** adlı bir yardımcı metot ekleyiniz.

1. **private** alanlar yalnızca tanımlandıkları sınıf içerisinde kullanılabilir.
2. **protected** yardımcı metot temel sınıfta ve türetilmiş sınıflarda kullanılabilir.
3. **public** property ve metotlar, sınıfın erişilebilir olduğu yerlerden kullanılabilir.
4. **internal** sınıflar aynı derleme birimi içerisinde kullanılabilir.

BEKLENEN ÇIKTI

terminal

```
GÜNCEL AKADEMİK BİRİM

Birim adı: Mühendislik Fakültesi
Personel sayısı: 10
Yıllık bütçe: 6.700.000,00 TL
```

03 İdari Birimin Oluşturulması

Üniversitenin idari birimlerini temsil etmek amacıyla **UniversityUnit** sınıfından türeyen **AdministrativeUnit** adlı bir sınıf oluşturunuz.

Harici çizim kapalı. Diyagram için düzenleyicide PlantUML iznini açın. UML kodu:

```
class AdministrativeUnit {
    - serviceCount : int

    + ServiceCount : int <<property>>

    + AdministrativeUnit(inputUnitId : int, inputUnitName :
string, inputStaffCount : int, inputAnnualCostPerStaff :
decimal, inputServiceCount : int)

    + CalculateAnnualBudget() : decimal <<override>>
}
```

BEKLENEN ÇIKTI

terminal

```
Birim numarası: 101
Birim adı: Mühendislik Fakültesi
Personel sayısı: 10
Yıllık bütçe: 6.700.000,00 TL

Birim numarası: 201
Birim adı: Öğrenci İşleri Daire Başkanlığı
Personel sayısı: 5
Yıllık bütçe: 2.500.000,00 TL
```

04 University Sınıfıyla Üniversite Raporunun Oluşturulması

Üniversiteye bağlı birimleri saklamak ve ortak rapor oluşturmak amacıyla **University** adlı bir sınıf tanımlayınız.

→ GEREKSİNİMLER / ADIMLAR

- **CalculateTotalBudget**, bütün birimlerin CalculateAnnualBudget metotlarını çağırarak toplam bütçeyi decimal türünde döndürmelidir.
- **CalculateTotalStaff**, birimlerin StaffCount değerlerini toplayarak toplam personel sayısını int türünde döndürmelidir.
- **DisplayReport**, üniversite adını, birim sayısını, birimlerin bilgilerini, toplam personel sayısını ve toplam yıllık bütçeyi göstermelidir.

Harici çizim kapalı. Diyagram için düzenleyicide PlantUML iznini açın. UML kodu:

```
class University {
    - universityName : string
    - units : UniversityUnit[]

    + University(inputUniversityName : string, inputUnits :
UniversityUnit[])

    + CalculateTotalBudget() : decimal
    + CalculateTotalStaff() : int
    + DisplayReport() : void
}
```

BEKLENEN ÇIKTI

terminal

```
ÜNİVERSİTE BİRİM RAPORU

Üniversite: Yeni Ufuk Üniversitesi
Toplam birim sayısı: 2

1. BİRİM

Birim numarası: 101
Birim adı: Mühendislik Fakültesi
Personel sayısı: 10
Personel başına yıllık maliyet: 600.000,00 TL
Yıllık bütçe: 6.700.000,00 TL

2. BİRİM

Birim numarası: 201
Birim adı: Öğrenci İşleri Daire Başkanlığı
Personel sayısı: 5
Personel başına yıllık maliyet: 480.000,00 TL
Yıllık bütçe: 2.500.000,00 TL

ÜNİVERSİTE TOPLAMLARI

Toplam personel sayısı: 15
Toplam yıllık bütçe: 9.200.000,00 TL
```

✓ Kazanımlar

- Akademik ve idari birimlerin ortak özelliklerini abstract bir temel sınıfta tanımlayabilmesi.
- Abstract sınıftan doğrudan nesne oluşturulamamasını ve somut sınıfların rolünü açıklayabilmesi.
- Abstract metotları türetilmiş somut sınıflarda override kullanarak uygulayabilmesi.
- Normal metotlar ile abstract metotları aynı temel sınıfta kullanabilmesi.
- base(...) aracılığıyla temel sınıfın yapıcı metoduna veri aktarabilmesi.
- private, protected, public ve internal erişim belirleyicilerini uygun amaçlarla kullanabilmesi.
- Ortak hesaplama işlemlerini protected bir metotta toplayarak kod tekrarını azaltabilmesi.
- public bir üyenin, içerisinde bulunduğu internal sınıfın erişim sınırını genişletmediğini açıklayabilmesi.
- Akademik ve idari birim nesnelerini ortak temel sınıf türündeki bir dizide saklayabilmesi.
- Aynı metot çağrısının farklı nesnelerde farklı hesaplama kurallarını çalıştırmasını açıklayabilmesi.
- Kalıtım ilişkisi ile bir sınıfın başka nesneleri içerisinde tutması arasındaki farkı ayırt edebilmesi.
- Property doğrulamaları aracılığıyla geçersiz güncellemelerde nesnenin mevcut durumunu koruyabilmesi.



KAYNAK KOD

https://github.com/zcomert/YZM203